

AI for Economic Research: Dynamic Models, Language, and Agents

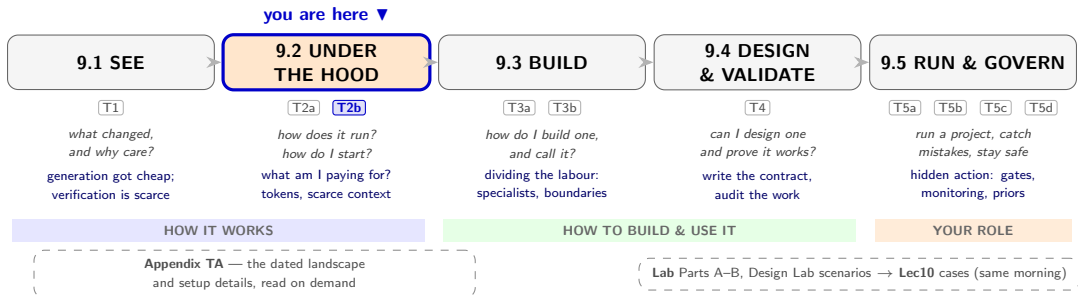
Lecture 9.2b: Your Setup — Files, Commands, First Session

Zhigang Feng

2026-09-12 20:41:49

Lecture 9 in One Map: Your Setup

This deck puts the loop on your machine: install it, learn its files and commands, and run a first session.



Route: terminal, install, and Git → the files an agent reads → the control surface → your first session → tokens and the context window. The economist's question, continued from T2a: tokens are the marginal cost, and the context window is the scarce input.

Terminal, Install, Git

The pieces, one install, and the safety net

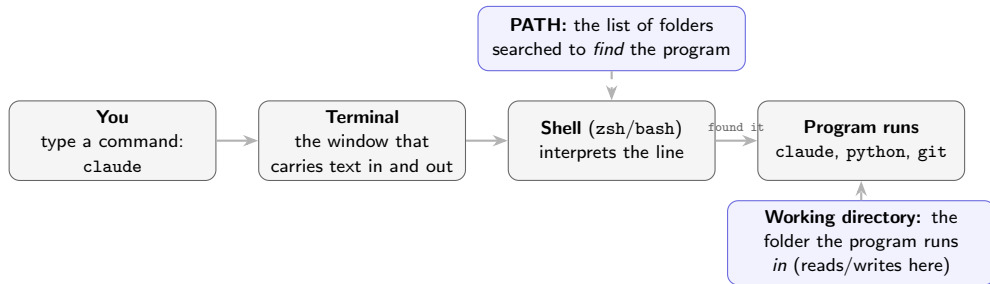
Terminal Vocabulary in 60 Seconds

Term	Meaning
Terminal	The text window where you type commands instead of clicking buttons.
Shell	The program that interprets commands. macOS often uses <code>zsh</code> ; many examples are written in <code>bash</code> .
Bash	A common shell language. When Claude Code says it will use Bash, it means “run this command in the terminal.”
Working directory	The folder commands run from. <code>cd my-project</code> changes it; <code>pwd</code> prints it.
PATH	The list of folders your shell searches when you type a command such as <code>python</code> or <code>claude</code> .

Why this matters: terminal agents act through shell commands. If the working directory or PATH is wrong, the agent may be competent but operating in the wrong place.

How the Terminal Pieces Fit Together

One picture for the five words: you type into the terminal; the shell finds the program via **PATH** and runs it in the working directory.



- When you type `claude`, the shell finds it via **PATH** and starts it *in your current folder* — that is why `cd my-project` comes first
- The agent's Bash tool is this same picture one level down: the agent typing into the same shell, in the same working directory

Install Once: Three Durable Steps

Setup has exactly three steps, and only the first command ever changes.

1. **Install the CLI** — one installer command for your OS (native installer script; Homebrew and npm also work):

```
curl -fsSL https://claude.ai/install.sh | bash
```

2. **Authenticate once** — run `claude`; a browser window opens; log in with your account or API key
3. **Work from the project** — every session starts the same way:

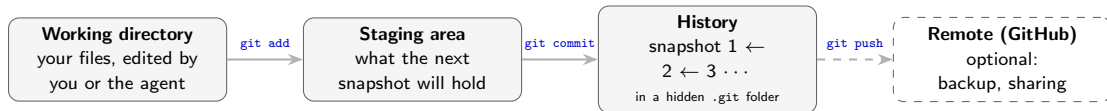
```
cd my-project  
claude
```

Install once per machine. Authenticate once per account. Initialize every session per project — automatically (next frames).

Durable vs. dated: the same three steps hold for Codex CLI and Gemini CLI. Exact commands, subscription tiers, and the Windows/OneDrive caveats are dated detail → Appendix TA and the official docs.

Git Is the Safety Net: A Time Machine for Agent Work

Git keeps snapshots of a project folder. An agent edits fast and wide; a snapshot before each session is your undo, your diff, and your audit trail.



Without it

- a terminal agent edits dozens of files in one session
- OneDrive and Dropbox sync by copying — they race the agent's rapid writes
- no undo, and no record of what changed or when

With it

- `git diff`: exactly what the agent changed, before you accept it
- `git restore .`: discard every uncommitted change — the emergency brake
- `git log`: every snapshot, with a time and a message; all of it offline

Practical rule: code in Git, large data wherever you like — but never let a sync service mirror a live `.git` folder; the two sync models collide and can corrupt the history.

Git Setup: A Four-Step Roadmap (Hand It to Your Agent)

You do not need to memorize commands. You need the roadmap — any LLM can walk you through each step.

Step 0 — check the install:

```
git --version
```

(if missing: git-scm.com or your package manager)

Step 1 — identify yourself (once per machine):

```
git config --global user.name "Your Name"  
git config --global user.email "you@edu.cn"
```

Step 2 — start the project (once per project):

```
cd my-project  
git init  
echo "data/ output/ *.pdf" > .gitignore  
git add . && git commit -m "initial setup"
```

Step 3 — the session rhythm (every session):

```
git add -A && git commit -m "pre-session"  
# ... agent works, you validate ...  
git add -A && git commit -m "validated: ..."
```

Emergency brake: `git checkout -- .` undoes *all* uncommitted changes instantly. And note: this setup is itself a perfect first agent task — paste the roadmap into the agent and say *“walk me through steps 0–3 for this folder.”*

Files: Where the Agent Reads From

Global versus project, and what loads when you start

Files First: Global vs. Local

Two layers of configuration. **Local overrides global.**

Path (Claude Code example)	Scope	What lives here
~/ .claude/settings.json	Global	Default permissions, model preference
~/ .claude/CLAUDE.md	Global	Your cross-project preferences
~/ .claude/skills/	Global	Skills available in every project
~/ .claude/projects/.../memory/	Global	Auto memory: the tool's own per-project notes
<project>/CLAUDE.md	Local	Project brain, read every session
<project>/ .claude/settings.json	Local	Project permissions (overrides global)
<project>/ .claude/skills/	Local	This project's slash commands
<project>/ .claude/agents/	Local	Specialized reviewer roles
<project>/ .claude/rules/	Local	Always-on guardrails

Other agents follow analogous layouts. Codex CLI uses AGENTS.md; Gemini CLI uses GEMINI.md. The pattern — global defaults plus a project-specific system prompt — is durable across products.

Rule: global dotfolder = your personal defaults (apply everywhere). Local dotfolder = the project's rules (apply only here). T5b fills the local layer with research content.

Project Files Across Harnesses

Every terminal agent reads the same kinds of files from your project. The names differ; the jobs do not.

Job	Claude Code	Gemini CLI	Codex CLI
Standing instructions	CLAUDE.md	GEMINI.md (name configurable) [†]	AGENTS.md [†]
Skills (SKILL.md)	.claude/skills/<name>/	.gemini/skills/<name>/ [†]	~/.codex/skills/<name>/ [†]
Custom sub-agents	.claude/agents/<name>.md	.gemini/agents/<name>.md [†]	a spawn_agent tool; custom types: not yet verified
Settings, permissions	.claude/settings.json	.gemini/settings.json [†] ; policy engine	~/.codex/config.toml [†] ; sandbox modes
Headless run (runc 3)	claude -p	gemini -p	codex exec

Trust is a setting, too. Gemini CLI ignores a project's agents, hooks, and skills until you trust the folder; Claude Code asks on first start; Codex wants a git repository. The lab's setup check tells you which case you are in.

Checked on 12 Sep 2026 (Claude Code 2.1.270, Gemini CLI 0.47.0, Codex CLI 0.154.0). [†] read from the installed package, not yet run.

Live examples: the repository's .claude/agents/, the lab's .claude/agents/, and the OLG demo's .claude/skills/.

Initialization: What Happens When You Type `claude`

Five steps run **before you type your first message** — each one loads a file from the previous frame:

1. **Read global config** — `~/.claude/settings.json`: permission defaults, model preference
2. **Detect project context** — searches current directory (and parents) for `CLAUDE.md` and `.claude/`
3. **Load project memory** — reads `CLAUDE.md` (and the tool's auto memory for this project) into the context window
4. **Auto-load all rules** — every `.claude/rules/*.md` file is loaded silently
5. **Ready** — context window pre-populated before you say anything

Analogy: steps 3–4 are a professor re-reading lecture notes before class. Students arrive; context is already loaded.

Cost warning: steps 3–4 consume tokens before you type anything. Keep `CLAUDE.md` and rules files concise.

Three Memories, Three Writers

Three different persistence ideas should not be confused.

Mechanism	Who writes it	Why it exists
CLAUDE.md	you / team	durable project instructions and conventions
Auto memory	the tool	per-project learned patterns and corrections
Optional course log	you / course	explicit progress notes such as progress.md

Course recommendation: rely on CLAUDE.md for shared rules, treat auto memory as helpful but not authoritative, and keep explicit logs when you want students to inspect the evolution of a project.

Where Those Four Agents Lived

Three different things get called “the agent”, and they have three different lifetimes. Only one of them is yours.

	Written by	Lives in	Survives?
the <i>type</i> (Explore, Plan)	the vendor, built in	the harness, delivered as a list in the system prompt	yes, but not yours to edit
the <i>brief</i>	the model , in the tool call	nowhere — four tool-call arguments	no
the <i>report</i>	the sub-agent	the message list, plus session scratch	no
code-reviewer, math-agent, ...	you	.claude/agents/ in the lab	yes, versioned

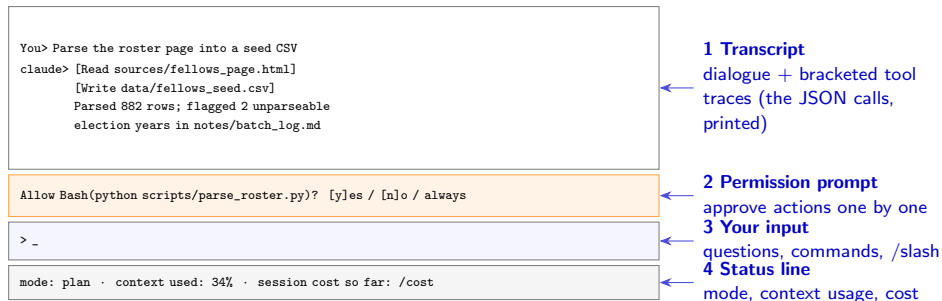
Where you put the folder decides whether the specialist exists. `.claude/agents/` resolves relative to the directory the session starts *in* — the global-vs-local rule above, applied to agents. Gemini CLI's `.gemini/agents/` works the same way, but only in a trusted folder.

The Control Surface

Commands, modes, keys, and permissions

The CLI at a Glance: One Screen, Four Zones

Everything you will ever look at in a terminal-agent session is one screen with four zones.



Zone 1's [Read ...] traces are the JSON tool calls of the earlier frame, printed for you — the audit trail is on screen by default. Layout details differ across products and versions; the four zones are durable.

Commands That Earn Their Keep (1): Framing and Steering

These six decide what the agent attempts — and how hard it thinks.

Command	What it does — and why it earns a place
Before the work starts	
/init	Writes a first CLAUDE.md from the repo — the brief that is re-read every session (T5b). Generating it beats writing it cold
/plan	Explores and drafts; executes nothing until you approve — so a wrong turn costs a re-read, not a git reset
How hard it thinks	
/model	Switches model mid-session, context preserved. A derivation and a variable rename do not deserve the same engine
/effort	Five levels, low → max: the biggest quality–cost dial you hold. Raise it for a proof or a subtle bug; lower it for mechanical edits
/fast	Toggles fast mode: same model, faster output — not a downgrade to a smaller one. For when latency, not depth, is the bottleneck
How you get words in	
/voice	Dictation (hold / tap / off). You specify a task far faster than you type it — and thin prompts are where these tools underperform

Spend effort on decisions, save it on execution: max effort on a long context can cost an order of magnitude more than the default.

Commands That Earn Their Keep (2): Context, Cost, and Undo

The other six manage your two budgets — the window and the meter — and undo mistakes.

Command	What it does — and why it earns a place
Mind the window	
<code>/context</code>	Draws current context use as a colored grid. Makes the invisible constraint visible: you see what is eating the window <i>before</i> quality degrades
<code>/compact</code>	Summarizes the conversation to free space. Keeps the thread, drops the detail — one you choose beats one that fires mid-derivation
<code>/clear</code>	New session, empty context; the old one stays on disk. The right move <i>between</i> unrelated tasks, which otherwise contaminate each other
Mind the meter — and the undo button	
<code>/usage</code>	Session cost, plan usage, activity (alias <code>/cost</code>). Turns an invisible meter into a number: check <i>before</i> a long agentic run, not after
<code>/rewind</code>	Restores files to an earlier checkpoint — the safety net that makes accept-edits mode reasonable
<code>/resume</code>	Reopens an earlier conversation. Research is not one sitting; yesterday's context is an asset, not something to re-explain

Four dials, not twelve commands. What it reads (`/context`, `/compact`, `/clear`) · how hard it thinks (`/model`, `/effort`, `/fast`) · how much it may do unasked (modes, `/permissions`) · what it costs (`/usage`). Not in the twelve: `/agents` (T3b).

Modes, Keys, and the Rest of the Surface

Commands are only one of three ways to steer. The other two are faster.

Modes — cycle with Shift+Tab:

Plan	Read-only: explores and drafts, executes nothing until you approve. Start every non-trivial task here
Accept-edits	Auto-approves routine file edits. Only after trust is earned
Default	Asks before each action

Keys:

Esc	Interrupt the agent mid-task, immediately
Esc Esc	Rewind to an earlier checkpoint
!	Run a shell command yourself; the output joins the context
@	Point at a file by path
Ctrl+R	Search your prompt history

Worth knowing, reached for less often:

/memory	Edit CLAUDE.md and memory settings directly
/permissions	Manage allow and deny rules — next frame
/status	Version, model, account, connectivity
/export	Save the conversation to a file: an audit trail of how a result was produced
/hooks	Inspect what runs automatically on tool events (T5b)
/doctor	Health-check the install when something is off

/help lists every command; typing / alone opens the menu.

Durable vs. dated: the four dials — context, effort, permission, cost — outlive any release. Exact command names, effort levels, and model aliases are dated detail → /help and the official docs.

Permissions: The Blast-Radius Dial

The agent's toolbox:

Tool	What it does
Bash	Run any shell command
Read/Grep	Read and search files
Write	Create or overwrite a file
Agent	Spawn a child instance

"Which Python ran this?"

- the agent's Bash inherits your shell's \$PATH: if you ran `conda activate myenv` before claude, *that* Python runs
- add a `which python` check to `CLAUDE.md`

The permission model:

- **Default:** `ask`. You see the exact command before it runs
- **Allowlist** trusted patterns in `.claude/settings.json`:

```
"permissions": {  
  "allow": ["Bash(python *)",  
            "Bash(git *)"],  
  "deny":  ["Read(.env)",  
            "Bash(rm -rf *)"] }
```

- **Bypass-permissions** skips all prompts: isolated sandboxes only

Think of authorizing a new RA. Default = every action needs your signature; allowlist = standing approval for routine work; deny = never allowed; bypass = handing over your own login. "*Blast radius*" = how much damage one wrong action could do before you notice.

One-Page Cheat Sheet: What You Type, Per Harness

The handful of commands the lab needs, in the three harnesses the course supports. Everything else is in each tool's `--help`.

To...	Claude Code	Gemini CLI	Codex CLI
start a session here	<code>claude</code>	<code>gemini</code>	<code>codex</code>
log in	<code>/login</code>	sign in on first start	<code>codex login</code>
run once, headless	<code>claude -p "..."</code>	<code>gemini -p "..."</code>	<code>codex exec "..."</code>
get machine-readable output	<code>--output-format json</code>	<code>-o json</code>	<code>--json</code>
stay read-only	<code>--permission-mode plan</code>	<code>--approval-mode plan</code>	<code>-s read-only</code>
see the context budget	<code>/context</code>	in the interactive UI; see each tool's docs	
run in the background	<code>claude --bg; claude agents</code>	—	<code>codex agents</code> (browse sessions)
check your setup	<code>python3 scripts/setup_check.py --live</code> (from the lab folder)		

From each CLI's own help on 12 Sep 2026 (Claude Code 2.1.270, Gemini CLI 0.47.0, Codex CLI 0.154.0); flags drift between releases — re-check with `--help`. [dated]

Your First Session

A real walkthrough on the fellows roster

First Session (1): Launch on a Real Project

Initialization in the abstract → what it actually looks like, on the fellows project from T1.

```
$ cd ~/research/es-fellows
$ claude
[reads ~/.claude/settings.json]
[reads ./CLAUDE.md]
[loads .claude/rules/provenance.md]
[ready]

You> Parse sources/fellows_page.html into data/fellows_seed.csv
      with columns: name, affiliation, election_year, profile_url.

claude> [Read sources/fellows_page.html]
        [Write data/fellows_seed.csv]
        Parsed 882 rows. Two election years did not parse
        (flagged in notes/batch_log.md). Spot-check 5 rows?
```

1. Initialization is automatic; no flags needed for the first session
2. The agent reads context (rules, references) *before* acting — visible as bracketed [Read ...] traces
3. Each trace is a JSON tool call that the harness translated into a real disk operation
4. Note the agent's last line: a competent session *ends turns with a checkable claim*, not just “done”

First Session (2): Pen and Paper, Then Plan Mode

0. Before the agent—pen + paper

Write one sentence: *"I want to compute X using method Y for question Z ."*

Fellows version: "a provenance-aware seed roster of all current ES Fellows, from the official page only."

1. Enter plan mode (Shift+Tab)

Describe your goal. The agent drafts a plan.
Nothing runs yet.

2. Review and approve

Read the plan. Ask questions. Revise. Then approve it.

Why this matters: plan review is the cheapest place to catch a wrong task definition.

Common beginner mistake: skipping Step 0.

Without a clear goal, the agent will confidently build the wrong thing.

What exactly to check before approving a plan: T5a gives the five plan-grading criteria.

First Session (3): Implement, Review, Iterate

3. Implement

The agent executes the approved plan. Press Esc to stop if wrong.

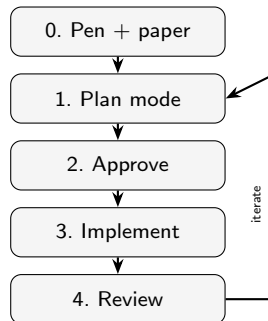
4. Review output

Does it make economic sense? Re-derive one number by hand.

Fellows version: open 5 random rows; follow each source URL.

5. Iterate

Fix, extend one feature, compact context, and return to step 1.



Practical takeaway: after each implementation pass, force one human check before you ask for the next extension.

What the First Session Leaves Behind

Close the terminal. What survives is not the conversation — it is the project state on disk.

In the folder now:

- `data/fellows_seed.csv` — the artifact (882 rows)
- `notes/batch_log.md` — what parsed, what was flagged
- `CLAUDE.md` — two new lines: the parse rules you settled on
- `.claude/rules/provenance.md` — the source rule, now always-on

Why this is the whole game:

- chat history is session-bound (resumable, but do not rely on it); **files persist and compose**
- next session, initialization reloads all of this *automatically* — batch 2 starts where batch 1 ended
- “the project files hold the rules and state across sessions” — the public-web analog of `CLAUDE.md` (Lec10 T4)

A session is good when it improves the project state, not when the conversation felt smart.

Close the Loop: Reflect, Save, Shrink

The last five minutes of a session are worth more than the next fifty. Three habits.

1. **Reflect.** End with a two-minute review: what worked, what you corrected, what surprised you. Ask the agent itself to summarize the session's decisions — it is cheap and surprisingly useful
2. **Save.** Write that summary into the plan or progress file, and update `CLAUDE.md` while it is fresh. The next session then starts from the updated plan, not from anyone's memory
3. **Shrink.** Prefer several small, focused sessions over one giant thread: a small context is cheaper, and the model's recall is sharper (mechanics in the next section). Break the big project into bounded sessions — *one intermediate product per session*

A big project is a sequence of small sessions, each ending with an updated plan. T5b turns this rhythm into a management system.

Budget

Tokens, cost, and the context window

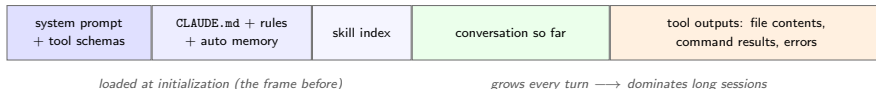
Tokens and Cost on the Radar

Every agent turn is a model API call. Cost scales with input and output tokens.

- **Input tokens.** Your prompt, conversation history, CLAUDE.md, rules, retrieved chunks. Every step that adds context adds cost.
- **Output tokens.** Generated code, JSON tool calls, prose explanations. Reasoning-heavy modes also produce many internal tokens.
- **Long sessions accumulate context.** Watch the status line; check /cost before long tasks.
- **Pin for reproducibility.** Record the model version, prompt template, and seed when applicable; the AEA journals already ask you to describe AI use at submission (T5d quotes the rule).

The Context Window Is the Agent's Working Memory

Everything the model “knows” during a turn sits in one finite window (Lec07 T3b). Here is what fills it in an agent session:



- The window is a **finite budget**: model-dependent, but always exhaustible by a long session
- Recall is not uniform — material buried mid-window is recalled worst (“lost in the middle,” Lec07 T3b); we do not reattach it, we *manage around* it
- Consequence: **long sessions degrade** — the agent slowly forgets early decisions and corrections

Compaction: What `/compact` Keeps and What It Drops

When the window fills, the harness summarizes: that is compaction. It is a lossy operation, and you should treat it as one.

- `/compact` = **summarize and clear**: older tool outputs are dropped first, the dialogue is condensed into a summary, and `CLAUDE.md` + rules are re-read from disk
- **What survives**: the decisions, the current task, key snippets. **What often does not**: the *reasoning* behind decisions, rejected alternatives, small corrections you made in passing
- **Auto-compaction**: by default the harness compacts on its own as the window fills — convenient, but the same information loss applies whether you triggered it or not
- `/clear` = **full reset**: conversation gone, files and memory remain; right choice when switching to an unrelated task

Rule: state belongs in files, not in the conversation. If a decision matters tomorrow, write it to `CLAUDE.md` or a progress note *before* the window fills — then compaction costs you nothing.

*T5c returns to what this trade-off costs research integrity. Dated (Claude Code, Sept. 2026): `/compact` takes focus instructions ("`/compact keep the asset-grid decision`"; `--autocompact` sets the window it compacts at (*auto*, or 100k–1M tokens); "`compact every 20 turns`" is folklore — compact between phases.*

Bridge: From Using an Agent to Building One

You can install, launch, steer, budget, and read the screen. In the lab folder sit six agent files you have not yet written.

- **Next (Part 9.3):** build one from scratch (T3a), then learn the three ways to call it (T3b)
- **The economist's version of Part 9.3:** how should the labour be divided — and what does each specialist need to know, and not know?